

コンサルティング結果要約

2026-04-22

コンサルティング結果要約

目的

CONSULTING.md に基づき、Qt アプリケーション開発の各開発実行段階で生成 AI をどう使い、人がどこで介入すべきかを整理する。

対象

- request -> requirements
- requirements -> architecture
- architecture -> design
- design -> implementation
- implementation -> verification
- verification -> foundation

全体要約

- 上流工程では、生成 AI は文書の構造化、観点補完、欠落候補の抽出に強い。
- 中流工程では、生成 AI は仕様から設計要素への写像や、設計粒度の分解支援に有効である。
- 下流工程では、生成 AI はコード、テスト観点、検証手順、運用文書のたたき台生成を高速化できる。
- 一方で、優先順位、採否判断、仕様確定、顧客合意、最終受け入れ判断は人が担うべきである。
- 高精度化には、入力文書の固定化、テンプレート化、段階ごとの分割実行、レビュー観点の明示が重要である。

適用条件

- 各工程で必要となる入力資産が、その工程を進めるのに足る範囲でそろっている
- 対象の Qt 版、使用モジュール、対象 OS、ビルド方法が、必要になる工程では明確である
- 生成結果をレビューして確定する担当者がある
- 既存保守案件では、既存コード、既知不具合、ビルドログなどの保守情報を参照できる

非適用条件

- 要求や設計が口頭中心で文書化されていない
- Qt の版や対象環境が曖昧である
- AI 出力を無審査で採用する前提である
- 機密データやログを外部 LLM へ送れないのに代替運用が未整備である

工程別の見方

- request -> requirements
 - 要望の整理と不足情報の抽出を中心に AI を使う
- requirements -> architecture
 - 要件を責務、構成、境界に変換する支援に AI を使う
- architecture -> design
 - モジュール責務を UI、描画、状態、入出力仕様に落とす支援に AI を使う
- design -> implementation
 - 採用した UI 技術方針に沿って実装と必要なテスト資産へ変換し、Qt AI Assistant 、 GitHub Copilot plugin for Qt Creator 、 Qt Creator MCP server を使って実装ループを補助する
- implementation -> verification

- テスト観点抽出、テストコード実装、検証実施を分けて整理し、単体寄りには `QTest`、GUI 回帰には `Squish` for Qt、保守と解析には `Squish AI Assistant` を組み合わせる
- `verification -> foundation`
 - 検証結果を運用上の再発防止、文書改善、基盤整備へ戻す整理に AI を使う

人の介入が必要な代表領域

- 顧客要求や業務要件の真意確認
- 実現可否、コスト、優先順位の判断
- Qt 特有の設計妥当性とプラットフォーム差分の判断
- セキュリティ、品質保証、出荷可否の最終判断
- 組織運用へ落とすためのルール化と定着

Qt 固有の重点論点

- `QObject` の親子関係と所有権
- `signal / slot` の接続寿命と責務境界
- `thread affinity` と `event loop` 制約
- High DPI、フォント、IME、入力デバイス、OS 差分
- `QSettings`、`resource system`、`moc / uic / rcc` を含むビルド整合

AI と既存自動化手段の役割分担

- AI
 - 文書整理、設計分解、コード草案、テスト観点抽出、ログ要約を担う
- `Qt AI Assistant`
 - Qt / QML に寄った公式の開発支援を担う
- `GitHub Copilot plugin for Qt Creator`
 - Qt Creator 上での汎用コード補完を担う
- `Qt Creator MCP server`
 - ローカルの Qt Creator 文脈で `build`、`run`、`debug`、`Issues` 確認を補助する
 - 新しい機能であり、現時点では基本的な操作支援が中心である
 - 現状は限定的な補助用途に留め、将来の機能拡張と連携強化に期待する
- `QTest`
 - 単体寄りのロジック、Widget、`signal / slot` の検証を担う
- `Squish for Qt`
 - GUI 回帰、受け入れ寄りのシナリオ確認を担う
- `Squish AI Assistant`
 - Squish スクリプト改善、失敗解析、ログ読解を補助する
- `clazy / clang-tidy / CTest / Sanitizers`
 - Qt 特有警告、静的解析、決定論的テスト、実行時検査を担う
- 人
 - 採否判断、例外判断、品質基準の適用、出荷判断を担う

推奨する進め方

- 各工程の入力文書を固定し、次工程に渡す情報を明確化する
- 一度に大きく生成させず、段階ごとに生成とレビューを分ける
- 生成結果には根拠、前提、未確定事項を明示させる

- 人は意思決定、例外判断、承認に集中する
- Qt Creator MCP server はローカルの build / debug 文脈取得に限定し、CI やリモート実行の代替にはしない
- Qt Creator MCP server はリリース直後であり、当面は基本機能を使った補助用途に限定する

導入段階

- PoC
 - 1 工程か 1 機能で文書生成、実装生成、検証補助の効果を測る
- 限定導入
 - QTest や Squish for Qt を一部機能へ適用し、Qt 特有のレビュー観点を定着させる
- 本格導入
 - テンプレート、プロンプト、KPI、レビュー規則、ログ蓄積、Qt Creator 運用を標準化する

KPI

- 文書初稿作成時間
- ビルド一発成功率
- clazy / clang-tidy 指摘件数
- 再生成率
- テスト観点漏れ件数
- GUI 回帰失敗の解析時間
- 出荷前の重大不具合検出件数

顧客提案時の要点

- 生成 AI は工程全体を無人化するものではなく、人の判断を前提にした半自動化に向く
- 文書テンプレートとレビュー規律を整えるほど、再現性と品質が上がる
- Qt のような UI と描画を含む開発でも、上流から下流まで一貫した支援が可能である
- 効果は工数削減だけでなく、抜け漏れ低減、レビュー観点の標準化、提案品質の平準化にもある